

Resolution: Implementation

Video Lesson

Duration: 00:08:25

Slide Deck

M09S09 (<https://canvas.tamu.edu/courses/430127/files/92570681?wrap=1>) 

Lesson Transcript

In this lecture, we will look at the full-resolution algorithm to be used in a fully automated theorem proving.

Level saturation resolution method (or two-pointer method)

Generate all possible resolvents:

- Generate sequences of clause sets $S^0, S^1, S^2, \dots$, where
$$S^0 = S \quad (\text{original set of clauses})$$
$$S^n = \{ \text{all possible resolvents of clauses} \\ C_1 \in (S^0 \cup \dots S^{n-1}) \text{ and } C_2 \in S^{n-1} \}$$
- This is basically a **breadth first search** method, and it can be extremely inefficient except for small problems.
- The problem is that **irrelevant** derivations are made: in generating an n-step proof, we also generate **all possible derivations** of n-1 steps.

The most basic resolution algorithm is called the level saturation resolution method or the two-pointer method.

Level saturation resolution method (or two-pointer method)

Generate all possible resolvents:

- Generate sequences of clause sets $S^0, S^1, S^2, \dots$, where
$$S^0 = S \quad (\text{original set of clauses})$$
$$S^n = \{ \text{all possible resolvents of clauses} \\ C_1 \in (S^0 \cup \dots S^{n-1}) \text{ and } C_2 \in S^{n-1} \}$$
- This is basically a **breadth first search** method, and it can be extremely inefficient except for small problems.
- The problem is that **irrelevant** derivations are made: in generating an n-step proof, we also generate **all possible derivations** of n-1 steps.



$$S^1 = C \in S^0 \cup \dots \cup S^0 = S^0 \text{ and } C_2 \in S^0$$

In this method, we start with the original set of clauses, including the negated conclusion at the end. From this, we want to create a series of sets of clauses, S^1, S^2 , up to S^n , until we derive **False**. So, S^n is a set of all possible resolvents of clauses, C_1 in S^0 union S^2 union S^{n-1} , and Clause 2 in S^{n-1} .

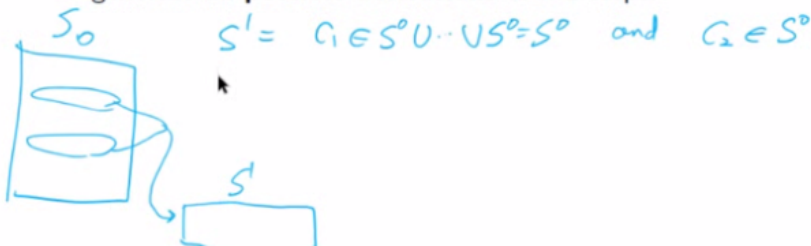
Let us see how S^1 should be constructed from the previously existing set of clauses S^0 . So, we

plug in zero here. Then C_1 , Clause 1, should come from S^0 union ... union S^0 . So, this becomes S^0 itself. So, Clause 1 comes from S^0 , and Clause 2 also comes from S^{1-1} , which is S^0 . So, Clause 2 also comes from S^0 .

Level saturation resolution method (or two-pointer method)

Generate all possible resolvents:

- Generate sequences of clause sets $S^0, S^1, S^2, \dots$, where
 $S^0 = S$ (original set of clauses)
 $S^n = \{ \text{all possible resolvents of clauses } C_1 \in (S^0 \cup \dots \cup S^{n-1}) \text{ and } C_2 \in S^{n-1} \}$
- This is basically a **breadth first search** method, and it can be extremely inefficient except for small problems.
- The problem is that **irrelevant** derivations are made: in generating an n-step proof, we also generate **all possible derivations** of n-1 steps.

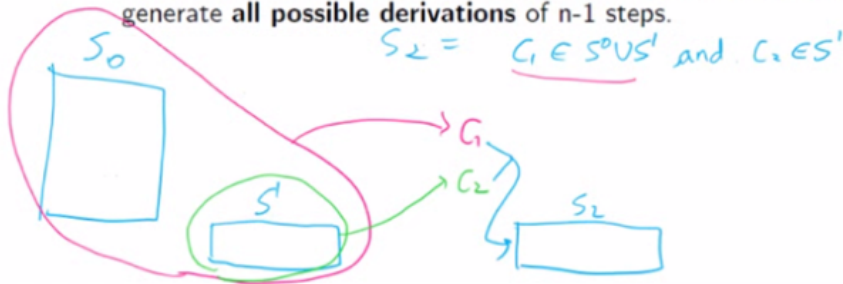


As shown here, S^1 is constructed by finding all possible resolvents from clauses in S^0 , and that will form a new set S^1 .

Level saturation resolution method (or two-pointer method)

Generate all possible resolvents:

- Generate sequences of clause sets $S^0, S^1, S^2, \dots$, where
$$S^0 = S \quad (\text{original set of clauses})$$
$$S^n = \{ \text{all possible resolvents of clauses} \\ C_1 \in (S^0 \cup \dots S^{n-1}) \text{ and } C_2 \in S^{n-1} \}$$
- This is basically a **breadth first search** method, and it can be extremely inefficient except for small problems.
- The problem is that **irrelevant** derivations are made: in generating an n-step proof, we also generate **all possible derivations** of n-1 steps.



116 / 130

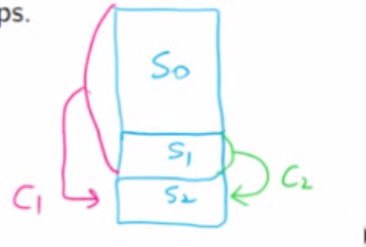
Moving on to S^2 . Now, S^2 comes from Clause 1 in S^0 union S^1 using this definition. And Clause 2 only comes from S^1 . So, as illustrated below, Clause 1 comes from the union of S^0 and S^1 , and Clause 2 comes from the newly generated subset from the immediately previous step, which will create another subset, S^2 .

We keep on creating S^3 and S^4 , and so on.

Level saturation resolution method (or two-pointer method)

Generate all possible resolvents:

- Generate sequences of clause sets $S^0, S^1, S^2, \dots$, where
 - $S^0 = S$ (original set of clauses)
 - $S^n = \{ \text{all possible resolvents of clauses } C_1 \in (S^0 \cup \dots \cup S^{n-1}) \text{ and } C_2 \in S^{n-1} \}$
- This is basically a **breadth first search** method, and it can be extremely inefficient except for small problems.
- The problem is that **irrelevant** derivations are made: in generating an n-step proof, we also generate **all possible derivations** of n-1 steps.

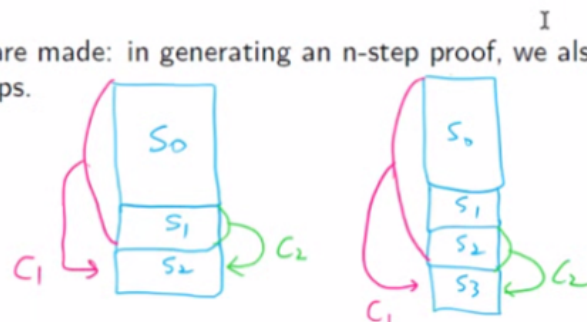


Another way of looking at it is like this. We start with S^0 , and then from that, we create S^1 , and S^2 is created from the union of S^0 and S^1 to get the first clause, and we get Clause 2 from S^1 .

Level saturation resolution method (or two-pointer method)

Generate all possible resolvents:

- Generate sequences of clause sets $S^0, S^1, S^2, \dots$, where
 - $S^0 = S$ (original set of clauses)
 - $S^n = \{ \text{all possible resolvents of clauses } C_1 \in (S^0 \cup \dots \cup S^{n-1}) \text{ and } C_2 \in S^{n-1} \}$
- This is basically a **breadth first search** method, and it can be extremely inefficient except for small problems.
- The problem is that **irrelevant** derivations are made: in generating an n-step proof, we also generate **all possible derivations** of n-1 steps.



Likewise, to construct S^3 , we take Clause 1 from S^0, S^1, S^2 , and Clause 2 from S^2 , and check

whether these two can be resolved. And only when they are resolvable, then we put that in S^3 .

This is basically a breadth-first search method. It can be extremely inefficient except for very small problems. The problem is that irrelevant derivations are made. So, in generating an n -step proof, we have to generate all possible derivations of $n-1$ steps.

Deletion Strategy

To reduce the huge number of generated clauses, we would like to delete clauses whenever possible. We can delete:

- ① Any tautology, e.g. $P(a) \vee \neg P(a) \vee Q(x)$.
- ② Any clause which duplicates an existing clause.
- ③ Any clause which is **subsumed by** an existing clause.

A clause C **subsumes** a clause D iff there is a substitution σ such that $C\sigma \subseteq D$ (recall that a clause can be represented as a set of literals). D is called a **subsumed clause**.

Deletion strategy will be complete if it is used with certain resolution algorithms (such as level saturation).

I

To improve the efficiency of the resolution algorithm, we commonly only use deletion strategies. Because a huge number of clauses will be generated, some of which may be equivalent, and so on. So, we want to remove these redundant clauses and clauses that are not needed.



There are three types.

Any clause that includes a tautology. Let us consider this clause $P(a) \vee \neg P(a) \vee Q(x)$, $P(a) \vee \neg P(a)$ reduces to **True** and **True**, or something is also **True**. So, this does not add to any value in the resolution process. So, we can remove these.

Second, is any clause that is an exact duplicate. Again, we can trivially remove these.

The last one is any clause which is subsumed by an existing clause.

A clause C subsumes a clause D if and only if there is a substitution σ such that when we apply the σ to the initial clause C , then the predicates in it are a subset of the predicates in the subsumed clause, D .

In this case, D is called the subsumed clause.

These deletion strategies will result in a complete proof process if it is used with certain resolution algorithms, such as the level saturation method that we have seen in the previous slide.

Subsumed Clause: Example (I)

Example:

- $C = P(x)$
- $D = P(a) \vee Q(a)$
- If $\sigma = \{x/a\}$, then
 $C\sigma = P(a) = \{P(a)\}$
 $\subseteq \{P(a), Q(a)\} = P(a) \vee Q(a) = D$.
- Since $C\sigma \subseteq D$, C subsumes D , and D can be deleted.

Here is an example of subsumed clauses. Let us say C is $P(x)$, and D is $P(a)$ or $Q(a)$. Because we can find the unifier σ to apply to Clause C , which will give us $P(a)$, and in the set notation is a single element $P(a)$ in the set.

And when we look at Clause D , and in set notation is $\{P(a), Q(a)\}$. Two items in the set. Because this is a superset of $\{P(a)\}$ or otherwise, $\{P(a)\}$ is a subset of $\{P(a), Q(a)\}$, then we can conclude that D is subsumed by C .

So, in this case, we can remove Clause D from the list of clauses. One way of looking at it is this. If this is **True**, then this becomes **True** or $Q(a)$, which evaluates to **True**. So, once this is **True**, it becomes **True**, and we do not need it.

Strategies to Improve Resolution

- ① **Deletion strategy:** remove tautology, duplicates, and subsumed clauses.
- ② **Unit preference:** resolve with clauses with the fewest literals.
- ③ **Set of support:** begin with set T consisting of the clauses from the **negated conclusion**. Each resolution step must involve a member of T , and the result is added to T .
- ④ **Linear resolution (Depth First):** Each step must be a resolution step involving the clause produced by the last step.

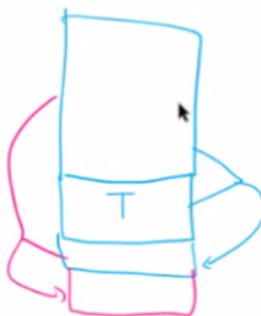
There are other strategies to improve resolution. We already saw the deletion strategy, removing

tautology, duplicates, and subsumed clauses. There is another method called "unit preference." In this approach, when we pick two clauses to resolve and put the resolved clause into the clause list, we prefer to resolve those with the shortest links, that is, clauses with the fewest literals.

Another approach is called the "set of support." We begin with set T consisting of the clauses from the negated conclusion, and each resolution step must involve a member of T or any clause that was derived from T .

Strategies to Improve Resolution

- ① **Deletion strategy:** remove tautology, duplicates, and subsumed clauses.
- ② **Unit preference:** resolve with clauses with the fewest literals.
- ③ **Set of support:** begin with set T consisting of the clauses from the **negated conclusion**. Each resolution step must involve a member of T , and the result is added to T .
- ④ **Linear resolution (Depth First):** Each step must be a resolution step involving the clause produced by the last step.



As shown here, if we have the clauses from the premises and clauses from the negated conclusion, every resolution step should involve either this set directly from the negated conclusion or some of these that are derived from it and something else.

Strategies to Improve Resolution

- 1 **Deletion strategy:** remove tautology, duplicates, and subsumed clauses.
- 2 **Unit preference:** resolve with clauses with the fewest literals.
- 3 **Set of support:** begin with set T consisting of the clauses from the **negated conclusion**. Each resolution step must involve a member of T , and the result is added to T .
- 4 **Linear resolution (Depth First):** Each step must be a resolution step involving the clause produced by the last step.

Finally, there is "linear resolution." This is similar to a depth-first search. In this case, each step must be resolved using the clause that was generated in the very last step, the previous step, or the immediately previous step.

Now, we are done with everything about resolution, including the theorem-proving method and how to implement these methods. In the remaining lectures, we are looking into the pros and cons of resolution, and also a concrete example where the resolution theorem proving can be applied.